

第24回 実践：Web ゲームプログラミング3（弾の発射）

実習時のフォルダ構成

Web_名前 の中に 24 フォルダを作る。第 23 回の `sample` フォルダと `exercise` フォルダを、24 の中にコピーして、続きから作る。

```
Web_名前
└── 24
    ├── sample          ← 第 23 回の sample をコピーする
    │   ├── index.html
    │   ├── style.css
    │   └── main.js
    └── exercise        ← 第 23 回の exercise をコピーする
        ├── index.html
        ├── style.css
        └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、`sample/main.js` に書き足す）
- `exercise`：自分のシューティングゲーム（演習 1～2、発展）

講義内容

`sample/main.js` に書き足す。書いたら `sample/index.html` をブラウザで開いて確認する。

1. 前回の復習

- `addEventListener("keydown", function (event) { })` で、キーが押されたときの処理を登録する。
- キーが押されているかどうかを `isLeftPressed` などの変数に記録し、実際の移動は `updatePlayer` で行う。

今回は、スペースキーで弾を撃てるようにする。弾は何発も同時に画面に出るので、第 19 回で学んだ配列を使う。

2. 今回の `main.js` の並び

```
// ===== 準備 =====
// ===== ゲームの状態 =====
// ===== 自機・弾・敵 ===== ← 見出しを変え、弾の変数を追加する
// ===== キー入力 ===== ← スペースキーを追加する
// ===== 更新 ===== ← updateBullets を追加し、update から呼び出す
// ===== 描画 ===== ← drawBullets を追加し、draw から呼び出す。drawUI に 1 行追加する
// ===== ゲームループ =====
```

3. 弾をオブジェクトの配列で表す

1 発の弾は、自機と同じように { x, y, w, h, speed } のオブジェクトで表す。何発もの弾を、1 つの配列に入れて管理する。

```
bullets = [ { x: 198, y: 440, w: 4, h: 12, speed: 8 },    ← bullets[0] (1 発目)
            { x: 198, y: 360, w: 4, h: 12, speed: 8 } ]  ← bullets[1] (2 発目)
```

- `bullets[0]` は 1 発目の弾（オブジェクト）。`bullets[0].y` で、1 発目の弾の y 座標が取り出せる。
- `for` 文の `i` を使うと、`bullets[i].y` のように、すべての弾を順に処理できる。

「自機」の部分の見出しを `// ===== 自機・弾・敵 =====` に変えて、弾の変数を 2 つ追加する。

```
// ===== 自機・弾・敵 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
let bullets = [];
let shotTimer = 0;           // 次の弾を撃てるまでのフレーム数
```

- `[]` は、要素が 1 つもない、空の配列。弾を撃つたびに、ここに弾を追加していく。
- `bullets` は、第 28 回で、空の配列を代入し直す（リスタートするとき）。そのため `let` で作る。

4. スペースキーを記録する

「キー入力」の部分を、次のように書き換える（第 23 回で書いた `console.log(event.key);` の行は、ここで消す）。

```
// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;
let isSpacePressed = false;

document.addEventListener("keydown", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
  if (event.key === " ") {
    isSpacePressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
  if (event.key === " ") {
    isSpacePressed = false;
  }
});
```

- スペースキーの `event.key` は " " (空白 1 文字)。

5. 弾を追加する・動かす・消す (updateBullets)

「更新」の部分の `updatePlayer` の下に、`updateBullets` を追加する。

```
function updateBullets() {
  if (shotTimer > 0) {
    shotTimer -= 1;
  }
  // スペースキーを押していて、撃てる状態なら、弾を追加する
  if (isSpacePressed && shotTimer === 0) {
    bullets.push({ x: player.x + player.w / 2 - 2, y: player.y, w: 4, h: 12, speed: 8 });
    shotTimer = 10;
  }
  for (let i = bullets.length - 1; i >= 0; i--) {
    bullets[i].y -= bullets[i].speed;
    // 画面の上に出た弾を消す
    if (bullets[i].y + bullets[i].h < 0) {
      bullets.splice(i, 1);
    }
  }
}
```

`updateBullets` は、次の 4 つのことを順に行っている。

(1) 連射の間隔 (shotTimer)

- `shotTimer` は、次の弾を撃てるまでの待ち時間（フレーム数）。
- 弾を撃つと 10 になり、1 フレームごとに 1 ずつ減る。0 になると、また撃てる。
- これがないと、スペースキーを押している間、毎フレーム（1 秒に 60 発）弾が出てしまう。

(2) `push` で弾を追加する

- 配列 `.push(値)`：配列の最後に、要素を 1 つ追加する。例：[10, 20] に `push(30)` をすると [10, 20, 30] になる。
- ここでは、新しい弾のオブジェクトを追加している。弾の `x` は、自機の中央 (`player.x + player.w / 2`) から、弾の幅の半分 (2) を引いた位置にする。

(3) 配列内の弾を動かす

- `for` 文で、すべての弾の `y` 座標を `speed` だけ減らす（上へ動かす）。第 19 回で、数値の配列 `bulletYs[i] -= 50` として練習したことを、オブジェクトの配列で行っている。

(4) 画面外に出た弾を `splice` で消す

- 弾の下端 (`bullets[i].y + bullets[i].h`) が 0 より小さくなったら、弾は画面の上に完全に出ていく。
- 配列 `.splice(番号, 1)`：その番号の要素を、1 つ配列から取り除く。例：[10, 20, 30] に `splice(1, 1)` をすると [10, 30] になる。
- 画面外の弾を消さないと、見えない弾が配列にたまり続け、処理がだんだん重くなる。

6. 削除しながらループするときは、後ろから調べる

`for` 文が `let i = bullets.length - 1; i >= 0; i--` になっていることに注意する。第 19 回で学んだ `i--` で、配列の**後ろから**順に調べている。

前から調べながら削除すると、次のように、調べずに飛ばしてしまう要素が出る。

手順	配列	起きること
1	[A, B, C]、 <code>i = 0</code>	A が画面外なので、A を削除する
2	[B, C] (B が番号 0 に詰まる)	<code>i++</code> で <code>i = 1</code> になり、C を調べる
3		B は一度も調べられない

- 後ろから調べると、削除して後ろの要素が詰まっても、それはもう調べ終わった要素なので、飛ばしがない。
- この授業では、「配列から要素を削除する `for` 文は、後ろから回す」ことをルールにする。

7. `update` と `draw` に追加する

「更新」の部分の `update` と、「描画」の部分で、次のようにする。`drawBullets` は `drawPlayer` の下に追加する。

```
function update() {
  frameCount += 1;
  updatePlayer();
  updateBullets();
}
```

```
function drawBullets() {
  ctx.fillStyle = "#ffff00";
  for (let i = 0; i < bullets.length; i++) {
    ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);
  }
}
```

```
function drawUI() {
  ctx.fillStyle = "#ffffff";
  ctx.font = "20px sans-serif";
  ctx.textAlign = "left";
  ctx.fillText(`FRAME: ${frameCount}`, 10, 30);
  ctx.fillText(`BULLETS: ${bullets.length}`, 10, 60);
}
```

```
function draw() {
  drawBackground();
  drawPlayer();
  drawBullets();
  drawUI();
}
```

- `drawBullets` : 描くだけで削除はしないので、前から順に回してよい。
- `drawUI` に、今ある弾の数 (`bullets.length`) を表示する行を追加した。弾を撃つと増え、画面の上に出ると減ることを確認する。

演習

`exercise/main.js` (自分のシューティングゲーム) に書く。

演習 1 弾を撃てるようにする

- 講義と同じように、スペースキーで弾を撃てるようにする。弾の色・大きさ・速さは、自分で決める。
- 弾が自機の中央から出るように、x 座標を調整する (弾の幅を変えたときは、引く値も変える)。
- 画面左上の「BULLETS」の数が、撃つと増え、画面の上に出ると減ることを確認する。

演習 2 動きを確かめる

- `shotTimer = 10;` の値を 3 や 30 に変えて、連射の間隔がどう変わるかを確かめる。最後に、自分がよいと思う値にする。
- `bullets.splice(i, 1);` の行の先頭に `//` を付けて、「BULLETS」の数がどうなるかを確かめる。確かめたら、`//` を消して元に戻す。

発展（任意）

- 1 回撃つと、自機の左端と右端から 2 発同時に弾が出るようにする。